Sistemas Operacionais

Resumo para Prova

Escalonamento de Processos & Threads

PARTE 1 — ESCALONAMENTO DE PROCESSOS

---

1. Objetivos do Escalonamento

O escalonador decide quem usa a CPU e quando. Seus objetivos são:

- Maximizar a utilização da CPU (mantê-la ocupada o máximo possível)
- Maximizar o throughput (número de processos concluídos por unidade de tempo)
- Minimizar o tempo de retorno (turnaround time)
- Minimizar o tempo de espera na fila de prontos
- Minimizar o tempo de resposta (primeira resposta ao usuário)

2. Preemptivo vs Não-Preemptivo

Tipo	Descrição	Exemplo
Preemptivo	O SO pode interromper um processo antes de terminar. Há custo de troca de contexto.	Round Robin
Não-preemptivo	O processo só sai da CPU quando termina ou bloqueia. Mais simples, sem overhead de preempção.	FIFO, SJF

3. Fórmulas Essenciais

Métrica	Fórmula
Tempo de retorno	Término − Chegada
Tempo de espera	Retorno − Burst (ciclos de CPU)
Tempo médio de espera	Soma dos tempos de espera ÷ número de processos

4. Algoritmos de Escalonamento

4.1 FIFO / FCFS — First In, First Out

- Executa na ordem de chegada, sem preempção
- Problema: efeito comboio — processos curtos ficam presos atrás de longos
- NÃO favorece processos curtos
- Maior tempo médio de espera entre os algoritmos

4.2 SJF — Shortest Job First

- Escolhe sempre o processo com menor burst disponível
- Matematicamente ótimo para minimizar o tempo médio de espera
- Versão não-preemptiva: ao liberar a CPU, escolhe o menor burst disponível
- Versão preemptiva (SRTF): interrompe se chegar processo com tempo RESTANTE menor
- Problema: pode causar starvation em processos longos

4.3 Prioridade

- Executa o processo com maior prioridade (menor número = maior prioridade no exercício)
- NÃO garante menor tempo médio de espera — SJF é melhor para isso
- Causa starvation: processos de baixa prioridade podem nunca executar
- Solução para starvation: aging (aumentar prioridade com o tempo de espera)

4.4 Round Robin (RR)

- Cada processo recebe um quantum de CPU; ao esgotar, volta para o fim da fila
- Favorece parcialmente processos curtos: os de burst < quantum terminam em uma rodada
- Quantum muito pequeno (próximo ao tempo de troca de contexto): overhead domina, sistema quase para
- Quantum muito grande (>> média dos bursts): comporta-se como FIFO
- No RR, tempo de espera = retorno − burst total (não por rodada)

4.5 Múltiplas Filas com Realimentação

- Todo processo começa na fila de maior prioridade (quantum pequeno)
- Se usar o quantum completo sem terminar, é rebaixado para fila de menor prioridade
- Processos curtos terminam nas filas altas; processos longos descem
- É o algoritmo que MAIS favorece processos curtos
- O mais flexível e o mais complexo de implementar

5. Comparativo dos Algoritmos — Exercício P1-P5

Processos P1(10), P2(1), P3(2), P4(1), P5(5) — todos chegam em t=0.

Algoritmo	Ordem de execução	Tempo médio de espera
FIFO	P1→P2→P3→P4→P5	9,6
SJF	P2→P4→P3→P5→P1	3,2 (melhor)
Prioridade	P2→P5→P1→P3→P4	8,2
Round Robin (q=3)	P1,P2,P3,P4,P5,P1,P5,P1	6,4

6. Starvation e Aging

Starvation: quando um processo espera indefinidamente porque sempre chegam processos com maior prioridade ou menor burst.

Quando ocorre: em algoritmos de Prioridade e SJF.

Solução — Aging: a prioridade do processo aumenta gradualmente com o tempo de espera, garantindo que ele seja escalonado eventualmente.

7. Problema do FCFS com Processo CPU-bound

Enquanto P1 (CPU-bound) usa a CPU, os dispositivos de I/O ficam ociosos. Quando P1 vai para I/O, a CPU pode ficar ociosa. Má utilização dos dois recursos.

Soluções: usar preempção (RR), priorizar processos I/O-bound, usar SJF.

PARTE 2 — THREADS

---

1. Threads vs Processos — Por que são mais leves?

Threads do mesmo processo compartilham: código, dados globais e arquivos abertos. Nada precisa ser duplicado.

Criar um processo exige alocar espaço de endereçamento separado, copiar recursos e criar estruturas de controle. Com threads, nada disso acontece.

Na troca de contexto: trocar processo invalida cache e TLB. Trocar threads do mesmo processo é mais barato — só salva PC, pilha e registradores da thread.

2. O que cada thread tem de próprio

Elemento	Por que é exclusivo
Contador de programa (PC)	Indica qual instrução a thread executará a seguir — cada thread pode estar em pontos diferentes do código
Pilha (stack)	Armazena chamadas de função, variáveis locais e endereços de retorno — sem pilha própria, duas threads na mesma função misturariam seus dados
Registradores	Estado do processador no momento da preempção

3. O que as threads compartilham (do processo)

- Código (segmento de texto)
- Dados globais
- Arquivos abertos
- Espaço de endereçamento

4. Benefícios do Multithreading

Benefício	Descrição
Responsividade	A aplicação continua respondendo ao usuário enquanto processa em background (ex: navegador renderiza e responde cliques ao mesmo tempo)
Aproveitamento de múltiplos núcleos	Threads podem executar em paralelo em núcleos diferentes, reduzindo tempo de execução
Economia de recursos	Criar e trocar threads é mais barato que processos

Compartilhamento de dados	Threads comunicam dados facilmente por memória compartilhada

5. Concorrência vs Paralelismo

Conceito	Descrição	Núcleos necessários
Concorrência	Múltiplas tarefas progridem intercaladas — o SO alterna entre elas rapidamente, dando ilusão de simultaneidade	1 ou mais
Paralelismo	Múltiplas tarefas executam verdadeiramente ao mesmo tempo, cada uma em um núcleo diferente	Obrigatoriamente 2+

Uma aplicação pode ser concorrente sem ser paralela, mas todo sistema paralelo é concorrente.

6. Threads de Usuário vs Threads de Kernel

Característica	Threads de Usuário	Threads de Kernel
Gerenciamento	Biblioteca do usuário (ex: pthreads)	Sistema operacional
Velocidade de criação	Rápida (sem chamada ao SO)	Mais lenta (exige chamada ao SO)
Bloqueio de I/O	Bloqueia TODO o processo	Bloqueia só a thread bloqueada
Paralelismo real	Não (kernel vê um único processo)	Sim (kernel escalona cada thread)

7. Modelos de Mapeamento

Modelo N:1 (Many-to-One)

- N threads de usuário → 1 thread de kernel
- Sem paralelismo real — mesmo com vários núcleos, só uma executa por vez
- PROBLEMA CRÍTICO: se uma thread bloquear, todas param (kernel bloqueia a única thread de kernel)
- Gerenciamento rápido pois fica no espaço do usuário

Modelo 1:1 (One-to-One)

- Cada thread de usuário = 1 thread de kernel
- Permite paralelismo real: kernel escalona threads em núcleos diferentes
- Se uma bloquear, as demais continuam normalmente
- Desvantagem: criar muitas threads tem custo maior
- Usado pelo Linux e Windows

Modelo M:N (Many-to-Many)

- M threads de usuário → N threads de kernel ($N \leq M$)
- Combina vantagens dos dois: cria threads com baixo custo E permite paralelismo real
- Se uma bloquear, redireciona outra thread de usuário para kernel disponível

- Mais flexível, porém mais complexo de implementar

8. Sincronização de Threads

synchronized

Garante que apenas uma thread por vez executa o método. A thread adquire um lock sobre o objeto; outras que tentarem entrar ficam bloqueadas até o lock ser liberado.

Evita condições de corrida: sem sincronização, duas threads podem ler e modificar o mesmo dado simultaneamente, produzindo resultados incorretos (ex: contador lido como 5 por duas threads, ambas escrevem 6 em vez do correto 7).

sleep() e join()

Método	O que faz	Exemplo de uso
sleep(ms)	Pausa a thread por um tempo determinado, liberando a CPU voluntariamente	Thread de monitoramento que lê temperatura a cada 5s chama sleep(5000) entre leituras
join()	Bloqueia a thread chamadora até que a thread alvo termine sua execução	Thread principal lança 4 workers e chama join() em cada um antes de somar resultados

9. POSIX Threads (pthreads)

Função	Papel
pthread_create()	Cria uma nova thread. Recebe: identificador, atributos, função a executar e argumentos.
pthread_join()	Faz a thread chamadora esperar que outra termine. Evita que o processo encerre antes das threads filhas.
pthread_exit()	Encerra a thread que a chamou sem encerrar o processo inteiro. Se usada na thread main, o processo continua vivo enquanto houver threads ativas.

RESUMO ULTRA-RAPIDO PARA REVISAO

Conceito	Em uma linha
FIFO	Ordem de chegada. Efeito comboio. Pior para processos curtos.
SJF	Menor burst primeiro. Melhor tempo médio de espera. Pode causar starvation.
Prioridade	Menor número = maior prioridade. Não garante menor espera. Causa starvation.
Round Robin	Fatias iguais (quantum). Quantum pequeno = overhead. Quantum grande = vira FIFO.
Filas c/ Realimentação	Melhor para processos curtos. Rebaixa processos longos.
Starvation	Processo nunca executa. Solução: aging.

Thread vs Processo	Thread compartilha memória do processo. Mais leve, troca de contexto mais rápida.
PC e Pilha	Cada thread tem o seu próprio. São o estado individual da thread.
N:1	Uma thread bloqueia? Todas param. Sem paralelismo real.
1:1	Paralelismo real. Linux e Windows usam. Custo maior por thread.
M:N	Melhor dos dois mundos. Mais complexo.
synchronized	Lock exclusivo. Evita condição de corrida.
Concorrência vs Paralelismo	Concorrência = intercalado (1 núcleo). Paralelismo = simultâneo (2+ núcleos).